Transcript QA Scoring System

Pipeline v3 (Laya) · repository nimjj/QA-system, branch main · built and tested in Docker from a fresh clone on 27 Oct 2026

1. Overview

A synchronous service that scores customer-support call transcripts against a configurable QA scorecard. A transcript is submitted over HTTP and the finished scorecard comes back in the same response. Deterministic rule checks handle the objective items; **Laya**, a small local yes/no decision model, handles the judgement calls. It runs on CPU inside a **single Docker container**, offline once its weights are on disk, with no queue, broker or LLM server. Each tenant's scorecard is read from PostgreSQL when it is reachable and from a built-in 11-item scorecard when it is not.

2. Architecture

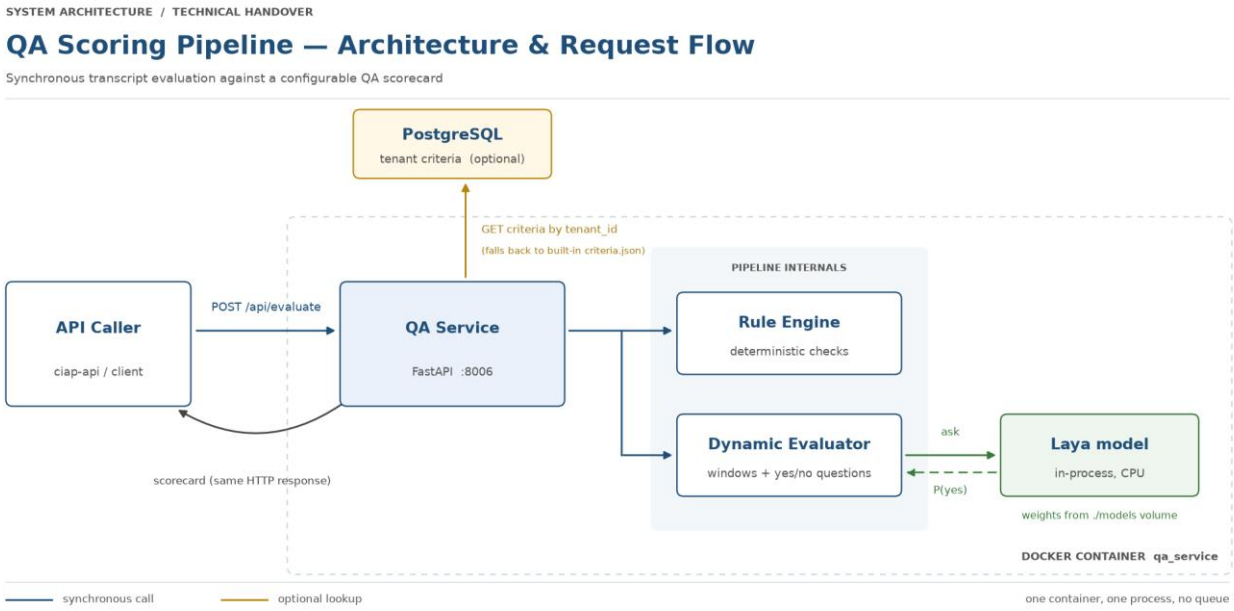


Figure 1 — Services, ports and request flow.

Service	Port	Responsibility
QA Service (FastAPI)	8006	Accepts transcripts, loads the tenant's scorecard, runs the rule engine and Laya, returns the scorecard. One container, one worker process.
Rule engine	—	In-process. Deterministic checks: branding, dead air, personalization, verification, repeated questions, profanity auto-fail.
Laya model	—	In-process, CPU. Answers a yes/no question per check. Weights are read from the ./models volume.
PostgreSQL (optional)	5432	Outside this container (the main backend's database). Per-tenant scorecard and on/off toggles: tables app.qa_* and connect_first.tenants.

No longer part of the system: Gateway, Redis, Celery worker, Ollama and config-db.

3. Request Flow

1. The caller sends POST /api/evaluate with the transcript and a tenant_id.
2. The service loads that tenant's active scorecard from PostgreSQL. If the database is unreachable or the tenant has no active items, it uses the built-in scorecard (src/v3/config/criteria.json) instead of failing. A criteria_data field in the request is ignored.
3. The transcript is validated and normalised (speaker roles, times, customer sentiment). An empty or unreadable transcript returns 422; a body over 2 MB returns 413.
4. Rule checks run first. A profanity auto-fail in an agent line ends the evaluation with score 0 and the model is skipped.
5. For the remaining items the service cuts a short window from the transcript and asks Laya a fixed yes/no question. Repeated identical windows are answered from an in-memory cache.
6. The service computes the score and returns the full scorecard in the **same HTTP response**. There is no job_id and nothing to poll.

Timing (measured, CPU): one 16–22 minute call takes about 5–30 seconds. The model answers one question at a time, so simultaneous requests share it: 4 simultaneous calls all returned after about 35 seconds. Callers should set a timeout of at least 120 seconds and keep concurrency low.

4. APIs & Payloads

Endpoint	Purpose	Key fields
POST /api/evaluate	Score a transcript (synchronous).	req: transcript[] or raw string, tenant_id, call_id, channel, customer_name → resp: call_id, tenant_id, status, result
GET /health	Liveness.	resp: status, service
GET /ready	Readiness: 200 once the model has loaded, 503 while loading or if loading failed.	resp: status, flow, model_ready
GET /api/tenants/{tenant_id}/criteria	Tenant scorecard with on/off flags.	resp: category_weights, categories[].line_items[]
PATCH /api/tenants/{tenant_id}/criteria/{line_item_id}/toggle	Switch one scorecard item on or off for a tenant (needs PostgreSQL).	req: {"is_active": true|false}
GET /api/samples	List bundled sample transcripts.	resp: array of transcript objects
GET/POST /api/tenants, /api/categories, /api/criteria (PUT/DELETE /api/criteria/{id})	Admin CRUD for tenants, categories and scorecard items (needs PostgreSQL).	see /docs (Swagger)

Removed since the Ollama version: GET /api/status/{job_id}, GET /api/criteria/{tenant_id} and POST /api/preview-prompt (they now return 404 or 405). If QA_API_KEY is set, every /api/ call must send it in the x-api-key header; unset leaves the service open. A complete request and response for POST /api/evaluate is on the next page (section 5).

5. REST Example: Request & Response

POST http://<host>:8006/api/evaluate · header Content-Type: application/json (plus x-api-key: <key> if QA_API_KEY is set) · answer 200 with the scorecard in the same response. This is a real call to the container, fine-tuned model, first request after start-up (4.78 s); the response is shown as received.

Request body


```
{"call_id":"smoke_001","tenant_id":"208","channel":"Call","customer_name":"Alex",
 "transcript": [
  {"speaker":"Agent","text":"Thank you for calling S-Net, my name is Sam. How can I help you today?","start_time_sec":0},
  {"speaker":"Customer","text":"My internet has been down since this morning and I am really
      frustrated.","start_time_sec":6,"sentiment_score":-35},
  {"speaker":"Agent","text":"I am so sorry about that, Alex, I understand how frustrating that is. First, can you confirm the PIN
      on your account?","start_time_sec":12},
  {"speaker":"Customer","text":"It is 4417.","start_time_sec":20,"sentiment_score":-20},
  {"speaker":"Agent","text":"Thank you, Alex, you are verified. What colour is the light on your router?","start_time_sec":26},
  {"speaker":"Customer","text":"It is flashing red.","start_time_sec":33,"sentiment_score":-15},
  {"speaker":"Agent","text":"Has anything changed recently, like a power cut or moving the router?","start_time_sec":39},
  {"speaker":"Customer","text":"No, nothing at all.","start_time_sec":46,"sentiment_score":-10},
  {"speaker":"Agent","text":"Thanks. I will reset your line profile from my side now. That is done.","start_time_sec":52},
  {"speaker":"Customer","text":"Okay, hold on.","start_time_sec":70,"sentiment_score":-5},
  {"speaker":"Agent","text":"Is it working now?","start_time_sec":85},
  {"speaker":"Customer","text":"Yes, the light just turned green and it works. Thank you so
      much!","start_time_sec":92,"sentiment_score":25},
  {"speaker":"Agent","text":"Great, so the issue is resolved. Thank you for choosing S-Net, have a great day!","start_time_sec":98}
 ]
}
```


Response body


```
{"call_id":"smoke_001","tenant_id":"208","status":"completed",
 "result": {
  "final_score":90.0,"auto_failed_count":0,"is_auto_fail":false,"auto_fail_reason":null,
  "category_summary":[{"category":"Soft Skills","passed":5,"total":5,"ratio":"05/05"},{"category":"Technical
      Knowledge","passed":5,"total":6,"ratio":"05/06"}],
  "scorecard":[
   {"category":"Soft Skills","name":"Branding and Survey Check","rating":"PASS","coaching":"Agent successfully used required
      opening and closing branding scripts.","evidence_turns":[0,12]},
   {"category":"Soft Skills","name":"Hold time and Dead Air","rating":"PASS","coaching":"Agent maintained active communication
      without excessive dead air (>30s)."},
   {"category":"Soft Skills","name":"Personalized the call/ticket appropriately","rating":"PASS","coaching":"Agent addressed
      customer by their verified name during the interaction.","evidence_turns":[2]},
   {"category":"Soft Skills","name":"Empathy & Acknowledgment Statement","rating":"PASS","coaching":"Agent demonstrated
      appropriate empathy and acknowledgment of the customer's situation."},
   {"category":"Soft Skills","name":"Build rapport and observed professionalism","rating":"PASS","coaching":"Agent maintained a
      professional, courteous, and respectful demeanor throughout."},
   {"category":"Technical Knowledge","name":"Paraphrasing","rating":"FAIL","coaching":"Failed to paraphrase core issue."},
   {"category":"Technical Knowledge","name":"Verified customer","rating":"PASS","coaching":"Agent verified customer identity
      within the required time window.","evidence_turns":[2]},
   {"category":"Technical Knowledge","name":"Probing","rating":"PASS","coaching":"Agent asked effective diagnostic questions to
      investigate the root cause."},
   {"category":"Technical Knowledge","name":"Took ownership of the problem","rating":"PASS","coaching":"Agent demonstrated clear
      ownership without deflecting or blaming other departments."},
   {"category":"Technical Knowledge","name":"Active listening","rating":"PASS","coaching":"Agent practiced active listening
      without repetitive or redundant questions."},
   {"category":"Technical Knowledge","name":"Confirmed the issue is resolved","rating":"PASS","coaching":"Agent explicitly
      confirmed that the issue was resolved before closing.","note":"resolved score 0.89 (cutoff 0.65); model 0.95, customer
      end mood 0.83"}
  ],
  "meta":{
   "flow":"v3","config_version":"v3.0-s1","model":"final-softce-3","al_second_source":false,"sentiment_available":true,
   "warnings":[],"gates":{"resolved":"v3","ownership":"v3","paraphrasing":"v2","empathy":"v2"},"model_rows":6,
   "model_words":413,
   "rows_by_check":{"empathy":1,"paraphrasing":1,"active_listening":1,"rapport":1,"probing":1,"resolved":1},"seconds":4.78
  }
 }
}
```


Fields. tenant_id picks the scorecard when PostgreSQL is set up (otherwise the built-in one). call_id is echoed back (generated if left out). customer_name feeds the Personalized check. sentiment_score goes on customer turns only (negative = upset). transcript may also be one string such as "[00:05] Agent: ...\n[00:12] Customer: ..." (no sentiment, so Empathy never triggers). evidence_turns are 0-based turn numbers; coaching is fixed text per check; final_score is 0–100. **Errors:** 401 wrong key, 413 over 2 MB, 422 unusable transcript, 503 model not ready, 500 on the PostgreSQL routes without a database. A 13-turn made-up call is only a format example: it is far shorter than the calls the model was tuned on, so its one FAIL says nothing about quality.

6. Scoring Model & Metrics

Each category starts at 100. For every line-item marked FAIL, its deduction_value is subtracted from that category (floored at 0). The final score is the weighted blend of category scores. A profanity auto-fail (agent lines only, or a FAIL in a category named "AUTO FAIL") forces the whole score to 0. Items switched off for a tenant are skipped and the weights are re-balanced.

Weights: Soft Skills 0.333, Technical Knowledge 0.667. **Final =** (SoftScore × 0.333) + (TechScore × 0.667).

Line item	Category	How it is evaluated	Deduction
Branding and Survey Check	Soft	Rule engine: opening and closing script for the brand S-Net	15
Hold time and Dead Air	Soft	Rule engine (timing gaps)	15
Personalized the call/ticket	Soft	Rule engine (uses the customer's name)	15
Empathy & Acknowledgment	Soft	Rule engine finds upset moments + Laya	35
Build rapport / professionalism	Soft	Phrase list + Laya (violation check)	20
Paraphrasing	Technical	Laya (yes/no on the agent's restatement)	15
Verified customer	Technical	Rule engine (PIN, address or security question within 4 min)	25
Probing	Technical	Laya on the agent's questions	25
Took ownership of the problem	Technical	Phrase list + Laya (violation check)	25
Active listening	Technical	Rule engine (repeated questions) + Laya	10
Confirmed the issue is resolved	Technical	Laya on the end of the call + the customer's closing mood	15

Worked example: a call that fails only "Verified customer" (−25, Technical) → Tech = 75, Soft = 100 → final = 100×0.333 + 75×0.667 = **83.3** (confirmed against the scoring code).

7. How Laya Decides (semantic checks)

Some checks cannot be matched by keywords — they need to know what was meant. For each of these, the service cuts a short window from the call and asks Laya one fixed yes/no question. Laya returns the probability of YES (0 to 1), and each check has its own PASS cutoff (for example Confirmed resolved passes at 0.65). Laya cannot write text, so the coaching line for every check is fixed. It runs locally on CPU: no calls leave the container, and the same input always gives the same answer.

Production runs the **fine-tuned checkpoint** final-softce-3 (808 MB). It is not in git and not in the Docker image: it is downloaded once from Google Drive (section 8) and mounted into the container, and docker-compose.yml points LAYA_CHECKPOINT=models/final-softce-3 at it. It also switches on the tuned cutoffs. If the weights are missing the service does not grade at all (503); it never falls back to another model. Measured on the four reference calls in inputs/Main tests, in a container started from a fresh clone:

Model	Scores (calls 1–4)	Items matching key
Reference answer key	100.0 / 88.3 / 56.7 / 13.3	—
Fine-tuned final-softce-3 (production)	100.0 / 88.3 / 50.0 / 30.0	42 of 44
Stock Laya (not used)	88.3 / 83.4 / 66.7 / 30.0	38 of 44

The reference key was written by the project, not by independent human reviewers, and no real call recordings have been tried yet. Treat scores as provisional.

8. Run & Ops Notes

- **Setup, in order:** (1) `git clone https://github.com/nimjj/QA-system.git`; (2) get the weights (next point); (3) `docker compose up -d --build` — service `qa-service`, container `qa_service`, port 8006, network `ciap-qa-network`. No `.env` file is needed. After a code change run `git pull` and the same command (source is copied into the image; the weights are not touched). Image about 1.8 GB.
- **Weights (required, once per machine):** download the Google Drive folder `final-softce-3` (Drive link, company Google account only), unzip it so that `models/final-softce-3/model.safetensors` exists, then check it: `docker compose run --rm --no-deps qa-service python Scripts/verify_weights.py`. The last line must be **ALL FILES MATCH**. Nothing is downloaded at run time (the container runs offline). Without the weights the service starts but `/ready` and `/api/evaluate` answer 503 with the reason.
- **Readiness and acceptance:** `GET /ready` returns 200 when the model is loaded (503 while loading, about 10–15 s). The container has a health check on it. Then run `docker compose exec qa-service python Scripts/try_call.py "inputs/Main tests/gold_01_excellent_22min.json"`: expect FINAL SCORE 100.0 and "model final-softce-3". Memory about 2 GB per container, CPU only. Full guide: `DEPLOYMENT_PLAN.md` in the repository.
- Config: tenant scorecards live in PostgreSQL (`app.qa_*`; set `DB_HOST DB_PORT DB_NAME DB_USER DB_PASSWORD`). Inside Docker the default `localhost` is the container itself, so point it at the database host (on Docker Desktop, `host.docker.internal`). The built-in scorecard, the brand (`s-net`) and the rule settings are in `src/v3/config/`.
- Limits to know: (a) if PostgreSQL is down the service silently grades with the built-in scorecard, so tenant toggles are ignored and the response does not say so; (b) the branding rule only recognises "thank you for calling/choosing S-Net", so another brand fails it until `brand` is changed; (c) there is no queue, so load is handled by the callers' concurrency and timeouts.
- Security and logs: optional `QA_API_KEY`, `CORS_ORIGINS` (default all), 2 MB request limit (`MAX_REQUEST_BYTES`). Logs go to `./logs/app.log` as JSON; request bodies and customer names are redacted unless `LOG_REQUEST_BODIES` is on.
- Tests: `python -m unittest discover -s tests/unit -t .` (96 tests, no model needed).

9. Changes Since the Ollama Version

Area	Before (Ollama)	Now (Laya)
Services	5 containers: gateway, Redis, worker, Ollama, config-db	1 container (qa-service); PostgreSQL optional
Calling it	POST then poll /api/status/{job_id}	POST /api/evaluate returns the result
Models	llama3.1 8B + nomic-embed-text (cosine ≥ 0.50)	Laya yes/no model; no embeddings
Scorecard	config-db MOCK_DB in code	PostgreSQL per tenant, built-in fallback
Scoring numbers	Weights 0.333 / 0.667, deductions as listed	Unchanged
Coaching text	Generated by the LLM	Fixed text per check

Tests-
https://docs.google.com/spreadsheets/d/1Rol__cQ73CrsDxtTkdzI_bFs0nvv9rWzj06usPXmjrY/edit?gid=1358365427#gid=1358365427